AV Autoencoder & Pose2OSC for GestureCap

GSoC 2026 Final Submission Report

Mathew Vallejo

https://github.com/mathewvallejo/GestureCap---GSoC-2026

This work was developed for the GestureCap project hosted by INCF for the 2026 Google Summer of Code (GSoC) session. GestureCap is dedicated to exploring the use of real time gesture data for control of digital sound parameters. Previous work has consisted of establishing a pipeline for using the MediaPipe hand tracking software, developing low-latency test protocols for real time performance, and the development of OSC-based interfacing between Python and visual programming languages such as Pure Data and Max/MSP.

The established goal for this GSoC contribution was to design and build a digital "Hypertheremin" that allows performers to control a digitally modelled theremin synthesizer via video input using the same (or similar) physical gestures used to control a traditional analog theremin instrument. This contribution focuses on the video input stage to the digital instrument. Hand tracking data obtained via the MediaPipe framework is used for initial hand recognition and fed down the gesture recognition pipeline, culminating in a mapping of gesture data to specific instrument controls. Gesture data is sent between the video input stage and the synthesizer engine via the Open Sound Control (OSC) protocol. The work is comprised of two separate components:

AV Autoencoder

The AV Autoencoder pipeline was developed to learn gesture structure from recorded theremin performances using calibrated MediaPipe hand-landmark data and aligned audio features. The system is organized into three Python stages: Camera Calibration and Preprocessing, Gated Recurrent Unit Autoencoder Training, and Gesture OSC Runtime.

In training, fixed-length motion windows are encoded into a latent gesture space while audio-derived log-mel features act as an auxiliary target, helping the model organize movement patterns around audiovisual performance structure. After training, the latent embeddings are clustered and exported as a runtime package. The live runtime uses motion landmarks only, assigns incoming gesture windows to learned clusters, and sends cluster, confidence, motion-energy, landmark, and latent-vector data over OSC for mapping to the Hypertheremin synthesizer.

Pose2OSC

Developed as a practical alternative to the autoencoder approach, Pose2OSC is a desktop application for performer-driven gesture enrollment and live OSC control. Users record examples of desired hand poses into a JSON gesture manifest, then the runtime extracts normalized MediaPipe hand-shape features and performs lightweight KNN recognition without requiring neural-network training. This provides a low-latency, explicit control path for the Hypertheremin synthesizer while preserving continuous landmark OSC output for expressive mapping in Max/MSP.

Current State

The current state of each component of the project is working end-to-end functionality, beginning with gesture input (training data for the AV Autoencoder, or user-captured gestures for a gesture manifest for Pose2OSC) and ending in OSC output with dedicated test patches for prototyping. The OSC output stage was developed in conjunction with a co-contributor responsible for building the audio synthesizer and includes a complete list of output messages and an example patch for OSC routing in Max/MSP.

Code Contributions

The contributions made for this project consist of individual Python project folders for each of the 3 stages of the AV Autoencoder (av_Camera_Calibration_Preprocess, av_GRU_autoencoder, av_Gesture_OSC_runtime) and a fourth folder containing the Pose2OSC code and application launcher (Pose2OSC). Each folder contains individual README.md files with full walkthroughs on how to set up the proper code environment, run the scripts, and contain all necessary information on dataflow between stages for the AV Autoencoder pipeline.

Remaining Work

The area of this work that would benefit most from targeted experiments and investigation is the collection and cleaning of video training data for the AV Autoencoder. It would be helpful to know how a model trained on data from one performer translates to use by another performer. This endeavor is in pursuit of a Hypertheremin that *learns its performer*—or at least common performance characteristics—as opposed to a performer needing to learn the new instrument.

In regard to the Pose2OSC approach, more experimentation with accomplished analog theremin players would provide insight as to how the gesture manifest might best be captured to best allow for an organic approach to the Hypertheremin from a playability standpoint. This involves decisions surrounding how many gestures to capture for complete control, and how those will be mapped to the synthesizer engine.

Challenges / Lessons

It was very interesting to unravel the web of design decisions for the development of this project. We had the opportunity to pursue multiple approaches to solving the same problem, determine what will work best under given circumstances, and ensure that all sides of the system can functionally integrate with each other. Integration between video/gesture input and the Hypertheremin synthesizer was a particularly satisfying problem to solve, as it was effectively the culmination of the work. The most challenging aspect of the project came from the AV Autoencoder design, which ultimately required being broken down into several stages to ensure clear data transferal between calibration, training, runtime export, and OSC output.